Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

Cyber Security RoadMap

Computer Networks

- What is Networking?
- Networking Components
- IP Addressing and MAC Address
- Networking Protocols and Ports
- 20 Common Network Protocols Explained
- Application Layer Protocols
- Transport Layer Protocols
- Network Layer Protocols
- Data Link Layer Protocols
- Security Protocols
- File Sharing and Directory Services
- Network Address Translation (NAT)
- Key Network Devices Router, Switch, Hub, Access Points, Firewall, Bridge
- What is the OSI vs TCP/IP Model, and can you explain each layer in detail?
- What is the difference between IPv4 and IPv6?
- What is DNS, and how does it work?
- What is a VLAN, and how does it enhance network security?
- VPN: Functions of a VPN, types of VPN, VPN tunneling protocol (PPTP, L2TP, SSTP, OpenVPN, WireGuard), SSH Tunnelling, VPN Topologies, VPN concerns, SOCKS Proxy, countermeasures
- What is the difference between TCP and UDP?
- What is the purpose of a proxy server?
- What is IPsec and how is it used in networking?
- Mutual Trust: mutual trust anatomy, Digital signature, Certificate Authorities (CAs) Cryptographic Key Management and Distribution, Kerberos and X.509
- IP Security: IP Security Overview, components of IPsec (AH, ESP, IKE), transport mode, tunnel mode, IP Security Policy, countermeasures
- Email security: Phishing and Spear Phishing, Business Email Compromise (BEC), Email Authentication Techniques (SPF, DKIM, DMARC), 2FA, Email Policies and Best Practices, Malware distribution, Malicious links attachments, MFA, Ransomware, Email Protocol (SMTP, STMPS, POP3, POP3S, IMAP, IMAPS), STARTTLS, Email Spoofing, PGP, countermeasures
- IDS/IPS (Intrusion Detection System / Intrusion Prevention System)

Shamsher Khan

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

Programming For Hackers

C Programming Topics

- Introduction to C
- History of C
- Features of C
- Application areas of C
- Execution flow of C program
- Other translators
- Structure of C program with example
- Keywords

Basic concepts

- Tokens
- Identifiers
- Constants
- Variables
- Data types
- Input and output functions
- Qualifiers
- Modifiers
- Escape sequences

Shamsher khan

Operators and Expressions

- Arithmetic operators
- Relational operators
- Logical operators
- Assignment operators
- Increment & Decrement operators
- Conditional/ternary operator
- Bitwise operator
- Size of operator

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

- Comma operator
- Operators' Precedence and associativity
- Expressions
- Evaluation of expressions

Control structures

- While
- For
- Do.. While
- Goto Statement
- Break and Continue Statement

Control/Decision-Making Statements

- Simple if
- if..else
- Nested if
- if..else ladder
- Switch..Case statement

Shamsher khan

Arrays

- Introduction to arrays
- Types of arrays
- 1d array
- 2d array (matrix)

string.h library

- strlen(str)
- strcpy(des_str, src_str)
- strcat(desc_str, src_str)

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

- strrev(str)
- strcmp(str1,str2)
- strlwr()
- strupr()

Functions

- Function types
- Built-in functions
- User-defined functions
- Recursive functions
- call by value and call by reference

Recursions

- Find the factorial of a given number by using a recursive function
- Find the sum of first n natural numbers by using recursion
- Find the gcd of two numbers by using recursion
- Find the root digit of the number by using recursion
- Program to calculate the power using recursion

shamsher khan

Storage classes

- Auto
- Static
- Extern
- Register

Pointers

- Pointer types
- Void pointer
- Null pointer

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

- Wild pointer
- Dangling pointer
- Array of pointers
- Pointer to pointer
- Pointer arithmetic

Dynamic memory allocation

- malloc()
- calloc()
- realloc()
- free()

File Handling

- Concept of a file
- Streams
- Text File and Binary Files
- Opening and closing files
- File input/output functions
- Formatted input-output functions
- Character input-output functions

Shamsher khan

Python Basics for Cybersecurity

- Python installation and setup
- Data types, variables, and operators
- Conditional statements and loops
- Functions and modular programming
- File handling (reading, writing, and manipulating files)

- Error handling and exceptions

Working with Python Libraries

- **Networking:** socket, ipaddress
- **Cryptography:** hashlib, cryptography, PyCrypto, Fernet
- **Web Interaction:** requests, urllib, BeautifulSoup (web scraping)
- **Data Handling:** json, csv, pickle
- **Regular Expressions:** re (for pattern matching)
- **Process Management:** subprocess, os

Structured query language (SQL)

Basics of SQL

- Introduction to SQL and relational databases
- SQL syntax and basic queries
- Data types in SQL
- SQL operators and functions
- SQL constraints (Primary Key, Foreign Key, Unique, Not Null)

Shamsher khan

2. SQL Operations

- Creating, modifying, and deleting databases and tables
- Data Manipulation Language (DML):
  - SELECT, INSERT, UPDATE, DELETE
- Data Definition Language (DDL):
  - CREATE, ALTER, DROP
- Data Control Language (DCL):
  - GRANT, REVOKE

- Transaction Control Language (TCL):
    - COMMIT, ROLLBACK, SAVEPOINT

3. Advanced SQL

- Nested and correlated subqueries
- Joins (INNER, OUTER, LEFT, RIGHT, CROSS)
- Grouping and aggregation (GROUP BY, HAVING, COUNT, AVG, SUM)
- Indexing and performance tuning
- Stored procedures and functions
- Triggers

PHP Programming

PHP Basics for Ethical Hacking

- **Syntax and Basic Constructs**
    - Variables, constants, and data types.
    - Control structures: if-else, switch, loops.
- **Functions**
    - Creating and using functions.
    - Understanding built-in PHP functions relevant to security.
- **Working with Forms**
    - Handling $_GET, $_POST, and $_REQUEST.
    - Validating user input securely.

Shamsher khan

2. User Input and Output Handling

- **Input Validation**
    - Sanitizing user input using filter_var(), htmlspecialchars(), and strip_tags().
- **Output Escaping**

- Preventing XSS with proper output escaping techniques.

- **File Handling**
  - Reading and writing files using fopen(), fread(), and file_put_contents().
  - Managing file uploads securely.

3. PHP and Databases

- **Database Interaction**
  - Connecting to databases using PDO or MySQLi.
  - Executing queries securely with prepared statements.
- **SQL Injection Testing**
  - Writing PHP scripts vulnerable to SQL injection.
  - Fixing vulnerabilities with parameterized queries.

shamsher khan

4. Sessions and Cookies

- **Session Management**
  - Using session_start() and $_SESSION.
  - Securing sessions with session_regenerate_id() and proper configurations.
- **Cookie Handling**
  - Setting and retrieving cookies with setcookie() and $_COOKIE.
  - Securing cookies with HttpOnly and Secure flags.

5. PHP File Inclusion

- **Dynamic File Inclusion**
  - Using include, require, include_once, and require_once.
  - Understanding and exploiting Local File Inclusion (LFI) and Remote File Inclusion (RFI) vulnerabilities.

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

6. Error and Exception Handling

- **Error Reporting**
  - Configuring error levels with error_reporting().
  - Logging errors instead of displaying them (ini_set('display_errors', 0)).
- **Exceptions**
  - Using try-catch blocks for secure exception handling.

7. PHP Security Features

- **Password Hashing**
  - Using password_hash() and password_verify() for secure password storage.
  - Avoiding outdated functions like md5() and sha1().
- **Data Encryption**
  - Encrypting data with PHP's openssl_encrypt() and mcrypt() (legacy).
  - Managing keys securely.
- **Cross-Site Request Forgery (CSRF)**
  - Generating CSRF tokens for form submissions.

Shamsher khan

8. Exploit Development

- **Custom Exploit Scripts**
  - Writing PHP scripts to test SQL injection, XSS, and file upload vulnerabilities.
- **Web Shells**
  - Writing and understanding PHP-based web shells.
  - Uploading and detecting web shells in controlled environments.

9. Advanced Topics

- **PHP Object Injection**
  - Understanding deserialization vulnerabilities with unserialize().

- Exploiting and patching object injection flaws.

- **Command Injection**
  - Using exec(), system(), and shell_exec() securely.
  - Detecting and preventing command injection.
- **Regular Expressions**
  - Writing secure regex patterns for input validation.

JavaScript

1. JavaScript Basics

- **Variables and Data Types**
  - var, let, and const.
  - Basic data types: strings, numbers, booleans.
- **Operators and Expressions**
  - Arithmetic (+, -, *, /), comparison (==, ===, !=, <, >), and logical (&&, ||, !) operators.
- **Control Structures**
  - if-else and switch statements.
  - Loops: for, while.

Shamsher khan

2. Working with the DOM

- **DOM Basics**
  - Accessing elements: document.getElementById(), document.querySelector().
  - Modifying content: element.innerHTML, element.value.
  - Changing styles: element.style.
- **Event Handling**
  - Adding events: addEventListener().

- Examples: onclick, onmouseover.

3. JavaScript and Forms

- **Capturing User Input**
  - Using document.forms and input fields.
  - Reading data from $_GET and $_POST with JavaScript.
- **Client-Side Validation**
  - Writing simple scripts to check required fields or validate email formats.
  - Learning how attackers bypass client-side validation.

4. Basic Exploits and Testing

- **Cross-Site Scripting (XSS)**
  - Understanding how input from forms or URLs can be reflected back to users.
  - Writing basic XSS payloads like <script>alert('XSS')</script>.
- **Cookie Access**
  - Reading cookies using document.cookie.
  - Basic understanding of session stealing and why it's dangerous.

Shamsher khan

5. Browser Console Basics

- **Using Developer Tools**
  - Opening the browser console (Ctrl+Shift+I or F12).
  - Running simple JavaScript commands directly in the console.
- **Dynamic Content Modification**
  - Changing text or images on a webpage using the console.
  - Example: document.querySelector('h1').innerText = 'Hacked!'.

6. File and Storage Access

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

- **Local Storage and Session Storage**
  - Storing and retrieving data with localStorage.setItem() and getItem().
  - Understanding the security implications of client-side storage.

7. Basic Fuzzing and Testing

- **Testing for Vulnerabilities**
  - Entering unexpected inputs (like '<script>') in forms.
  - Observing application responses to identify vulnerabilities.

8. Writing Simple Scripts

- **Dynamic Script Injection**
  - Writing and injecting a simple script into a vulnerable site.
  - Example: <script>document.body.style.backgroundColor='red';</script>.
- **Custom Alerts**
  - Writing basic alert boxes to simulate attacks: alert('This site is vulnerable!').

Shamsher khan

9. Understanding the Basics of APIs

- **Interacting with APIs**
  - Making simple fetch() requests to send or retrieve data.
  - Example: fetch('https://example.com/api').then(response => response.json()).then(data => console.log(data));.

10. Simple Ethical Hacking Exercises

- **Testing Input Fields**
  - Try submitting unexpected characters (<, >, ;, etc.) and observe responses.
- **Testing Cookies**

- Use document.cookie to view cookies and understand how they store session data.

Assembly Language programming

1. Introduction to Assembly Language

- **What is Assembly Language?**
  - Basics of low-level programming and how it interacts with hardware.
  - Understanding assemblers (e.g., NASM, MASM, or FASM).
- **Basic CPU Architecture**
  - Registers (e.g., EAX, EBX, ECX, EDX, etc.).
  - Program Counter (PC), Stack Pointer (SP), and Flags Register.
- **Instruction Set Basics**
  - Basic operations: arithmetic, logical, data movement.
  - Types of instructions: RISC (Reduced Instruction Set Computing) vs. CISC (Complex Instruction Set Computing).

Shamsher khan

2. Assembly Language Basics

- **Hello World Program**
  - Writing a simple "Hello World" program in Assembly to understand syntax.
- **Data Representation**
  - Binary, hexadecimal, and decimal formats.
  - Using immediate values, registers, and memory addresses.
- **Basic Instructions**
  - Data transfer: MOV, PUSH, POP.
  - Arithmetic: ADD, SUB, MUL, DIV.
  - Logical: AND, OR, XOR, NOT.

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

3. Working with Memory

- **Understanding Memory Segments**
  - Code, data, stack, and heap segments.
  - How memory is addressed in Assembly.
- **Addressing Modes**
  - Immediate, direct, indirect, and indexed addressing.
- **Stack Operations**
  - Using PUSH and POP.
  - Understanding the call stack and how functions use it.

4. Control Flow

- **Jump Instructions**
  - Unconditional jumps: JMP.
  - Conditional jumps: JE, JNE, JG, JL.
- **Loops**
  - Writing loops using labels and jump instructions.
- **Function Calls**
  - Using CALL and RET.
  - Passing arguments via registers or stack.

Shamsher khan

5. I/O in Assembly

- **Basic Input and Output**
  - Using system calls for basic input/output operations.
  - Example: Writing and reading strings to/from the console.
- **Interrupts**
  - Using interrupts (INT instruction) for I/O operations.
  - Common interrupts (e.g., INT 0x80 in Linux).

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

6. Assembly and Security

- **Buffer Overflows**
  - Writing programs to demonstrate how buffers can overflow.
  - Understanding how Assembly handles memory to prevent or exploit overflows.
- **NOP Sleds**
  - Using NOP instructions in shellcode for exploit payloads.
- **Shellcoding Basics**
  - Writing simple shellcode to execute basic commands.
  - Converting Assembly instructions into raw machine code.

7. Reverse Engineering

- **Disassembly**
  - Understanding how programs written in higher-level languages translate to Assembly.
- **Analyzing Binary Code**
  - Identifying functions, loops, and conditional statements in disassembled code.
- **Debugging**
  - Using tools like GDB (GNU Debugger) to step through Assembly code.

Shamsher khan

8. Tools for Assembly Programming

- **Assemblers**
  - NASM (Netwide Assembler), MASM (Microsoft Macro Assembler), or FASM (Flat Assembler).
- **Debuggers**
  - GDB, OllyDbg, or IDA Free.
- **Emulators**

- Using platforms like QEMU or online emulators to practice Assembly without hardware risk.

Linux Operating System

1. Introduction to Linux and its Kernel

- **What is Linux?**
  - History and evolution of Linux.
  - Open-source nature and benefits.
- **Linux Kernel**
  - Role and functions of the kernel.
  - Kernel modules.
- **Distributions of Linux**
  - Types of Linux distributions (e.g., Ubuntu, CentOS, Debian, Fedora).
- **Components of Linux**
  - Kernel, shell, file system, libraries, and utilities.

Shamsher khan

2. Accessing the Command Line

- **Terminal Basics**
  - What is a terminal/console?
  - Accessing the terminal in various Linux distributions.
- **Basic Commands**
  - Navigating the command line: cd, ls, pwd, echo.
- **Using the Bash Shell**
  - Overview of the Bash shell.
  - Shell prompt and user input.
- **Command Syntax and Structure**

- Options and arguments.
- Syntax of commands.

3. Managing Files from the Command Line

- **Navigating the File System**
  - Understanding the directory structure (/, /home, /etc).
  - Commands for navigating: cd, ls, pwd.
- **File Operations**
  - Creating files: touch, echo.
  - Listing files: ls.
  - Copying files: cp.
  - Moving files: mv.
  - Deleting files: rm.
- **Directory Operations**
  - Creating directories: mkdir.
  - Removing directories: rmdir.

Shamsher khan

4. Creating, Viewing, and Editing Text Files

- **Viewing Files**
  - cat, more, less, head, tail commands.
- **Editing Files**
  - Text editors: nano, vi, vim.
  - Basic file editing commands (e.g., copy, cut, paste).
- **Redirecting Output**
  - Output redirection (>, >>).
  - Input redirection (<).
  - Piping (|).

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

5. Managing Local Users and Groups

- **Adding Users**
  - Creating users: useradd, adduser.
  - Modifying user accounts: usermod, chsh.
- **Deleting Users**
  - Removing users: userdel.
- **Managing Groups**
  - Adding/removing groups: groupadd, groupdel.
- **User Permissions**
  - Understanding user types: root, regular users.
  - Changing user groups: gpasswd.

shamsher khan

6. Controlling Access to Files (Permissions)

- **Understanding File Permissions**
  - File permissions: r, w, x (read, write, execute).
  - Owner, group, and others.
- **Changing File Permissions**
  - chmod command.
  - Numeric mode vs symbolic mode.
- **Changing Ownership**
  - chown and chgrp.
- **Special Permissions**
  - Setuid, setgid, sticky bit.

7. Monitoring and Managing Linux Processes

- **Viewing Processes**

- ps, top, htop commands.
- Process states and details.

- **Managing Processes**
  - Stopping and pausing processes: kill, killall, pkill.
  - Changing process priority: nice, renice.
- **Process Hierarchy**
  - Understanding parent and child processes.
  - Process IDs (PID) and process trees.

8. Controlling Services and Daemons

- **Understanding Services and Daemons**
  - What are services and daemons?
  - Common Linux daemons (e.g., sshd, httpd, cron).
- **Service Management**
  - systemd basics.
  - systemctl commands to start, stop, restart, and enable services.
- **Checking Service Status**
  - systemctl status and journalctl for logs.
  - Using service for older systems.

Shamsher khan

9. Configuring and Securing SSH

- **Understanding SSH**
  - What is SSH (Secure Shell)?
  - Configuring SSH server (sshd).
- **Key-based Authentication**
  - Generating SSH keys: ssh-keygen.
  - Copying SSH keys: ssh-copy-id.

- **SSH Configuration**
  - Configuring sshd_config for better security.
  - Changing default SSH port, disabling root login.
- **Securing SSH**
  - Using firewalls (ufw, iptables).
  - Enabling SSH key authentication, disabling password login.

10. Analyzing and Storing Logs

- **System Logs**
  - Types of logs (e.g., authentication logs, system logs).
  - Common log files: /var/log/syslog, /var/log/auth.log, /var/log/messages.
- **Viewing Logs**
  - Using cat, tail, less, grep to analyze logs.
- **Log Rotation**
  - Managing log file sizes using logrotate.
- **Configuring Logging**
  - Adjusting logging levels and settings in /etc/rsyslog.conf.

Shamsher khan

11. Managing Networking

- **Network Configuration**
  - Basic networking commands: ip, ifconfig, hostname.
  - Configuring network interfaces.
- **Checking Network Connectivity**
  - ping, traceroute, netstat.
  - Checking open ports: ss, lsof.
- **Network Tools**
  - curl, wget, nc for testing and troubleshooting.

- **Firewall Configuration**
  - Basics of iptables and ufw for firewall management.

12. Archiving and Transferring Files

- **File Compression**
  - Creating compressed files: tar, gzip, zip, bzip2.
- **Extracting Files**
  - Extracting compressed files: tar -x, gunzip, unzip.
- **Transferring Files**
  - Using scp, rsync, ftp, sftp.
  - Transferring files securely over the network.

Shamsher khan

13. Installing and Updating Software Packages

- **Package Managers**
  - Using apt (Ubuntu/Debian), yum (RHEL/CentOS), zypper (openSUSE).
- **Installing Software**
  - Installing software: apt install, yum install, zypper install.
- **Updating Software**
  - Updating packages: apt update, yum update, zypper update.
- **Removing Packages**
  - Removing packages: apt remove, yum remove, zypper remove.

14. Accessing Linux File Systems

- **Mounting File Systems**
  - Understanding the /etc/fstab file.
  - Mounting drives: mount, umount.
- **File System Types**

- Common Linux file systems: ext4, xfs, btrfs.

- **File System Permissions**
  - Accessing and modifying file systems with proper permissions.

15. Scheduling Future Tasks

- **Cron Jobs**
  - Scheduling tasks with cron and crontab.
  - Understanding cron syntax for scheduling.
- **at Jobs**
  - Scheduling one-time tasks using the at command.
- **Systemd Timers**
  - Using systemd timers for recurring tasks.

16. Managing Basic Storage

- **Understanding Storage Devices**
  - Types of storage devices: HDD, SSD, partitions.
- **Partitioning and Formatting**
  - Using fdisk, parted to create partitions.
  - Formatting partitions: mkfs.ext4, mkfs.xfs.
- **Mounting and Unmounting**
  - Mounting file systems to specific directories.
  - Automating mount points in /etc/fstab.

17. Managing Logical Volumes

- **Logical Volume Management (LVM) Basics**
  - What is LVM? Physical volumes (PVs), Volume groups (VGs), and Logical volumes (LVs).

Shamsher khan

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

- **Creating Logical Volumes**
  - Creating PVs: pvcreate.
  - Creating VGs: vgcreate.
  - Creating LVs: lvcreate.
- **Managing Volumes**
  - Extending and shrinking volumes.
  - Resizing file systems and logical volumes.

Bash Shell Scripting

1. Introduction to Bash and Shell Scripting

1.1. What is Bash?
1.2. Shell vs Scripting
1.3. Basic Structure of a Shell Script
1.4. Writing and Running Your First Script
1.5. Making Scripts Executable

2. Basic Bash Commands and Concepts

2.1. Introduction to Basic Bash Commands
2.2. Working with Variables
2.3. User Input and Output
2.4. Command Substitution
2.5. File Redirection (Input, Output, and Errors)
2.6. Using Loops in Bash

3. Conditional Statements

3.1. If-Else Statements
3.2. Elif Condition
3.3. Case Statements
3.4. Comparing Strings and Numbers
3.5. Logical Operators in Conditions

4. Loops and Iteration

4.1. For Loops
4.2. While Loops

Shamsher khan

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

4.3. Until Loops
4.4. Break and Continue Statements
4.5. Nested Loops

5. Functions in Bash

5.1. Introduction to Functions
5.2. Defining Functions
5.3. Passing Arguments to Functions
5.4. Returning Values from Functions
5.5. Using Local Variables in Functions

6. File Handling in Bash

6.1. Reading and Writing Files
6.2. Checking File Existence
6.3. File Permissions and Ownership
6.4. Redirecting Output to Files
6.5. Appending to Files

7. Regular Expressions and String Manipulation

7.1. Introduction to Regular Expressions in Bash
7.2. Using grep with Regex
7.3. String Matching and Replacement
7.4. Advanced Text Processing with sed
7.5. Parsing Text with awk

Shamsher khan

8. Arrays in Bash

8.1. Defining and Accessing Arrays
8.2. Looping Through Arrays
8.3. Associative Arrays
8.4. Modifying Arrays
8.5. Multi-Dimensional Arrays

9. Process Management in Bash

9.1. Background Processes
9.2. Monitoring Processes
9.3. Killing Processes
9.4. Process Scheduling with cron
9.5. Managing System Resources

10. Networking in Bash

Email: Shamsher.khan.7362@gmail.com **LinkedIn:** https://www.linkedin.com/in/shamsher-khan404

10.1. Basic Networking Commands (ifconfig, ip, netstat)
10.2. Using ping and traceroute in Scripts
10.3. File Transfer with scp, rsync, and curl
10.4. Port Scanning with nmap and netcat
10.5. Interacting with APIs Using curl

11. Automating System Administration Tasks

11.1. User and Group Management
11.2. Backup and Restore Strategies
11.3. Log Management and Log Rotation
11.4. Software Installation Automation
11.5. System Health Checks

12. Error Handling and Debugging

12.1. Exit Status Codes and $?
12.2. Debugging Scripts with set -x
12.3. Error Handling with trap
12.4. Custom Error Messages and Logging
12.5. Using stderr and stdout for Debugging

13. Scripting Security and Best Practices

13.1. Secure Scripting Practices
13.2. Preventing Command Injection
13.3. Avoiding Race Conditions
13.4. Using sudo Safely in Scripts
13.5. Preventing Unauthorized Access

14. Advanced Bash Techniques

14.1. Working with Pipes and Filters
14.2. Handling File Descriptors
14.3. Signal Handling and Process Management
14.4. Complex String Manipulation
14.5. Integrating Bash with Other Languages (Python, Perl, etc.)

15. Automating Penetration Testing Tasks

15.1. Automating Vulnerability Scanning with nmap
15.2. Automating Reconnaissance Tasks
15.3. Writing Scripts for Exploit Testing

Shamsher khan

15.4. Cracking Passwords with Bash
15.5. Automating Post-Exploitation Tasks

16. Security Auditing and Penetration Testing Tools

16.1. Scripting with nikto, aircrack-ng, and Other Tools
16.2. Automating Information Gathering
16.3. Writing Simple Exploits
16.4. Social Engineering Attacks with Bash Scripts
16.5. Evading Security Detection Using Bash

17. Script Optimization and Performance Tuning

17.1. Optimizing Bash Scripts for Speed
17.2. Reducing Resource Consumption
17.3. Best Practices for Writing Efficient Code
17.4. Profiling Scripts for Performance
17.5. Memory Management in Shell Scripts

18. Bash Scripting for Ethical Hacking

18.1. Reverse Shells with Bash
18.2. Fileless Malware Scripting Techniques
18.3. Automating Exploit Execution
18.4. Writing Payloads in Bash
18.5. Evasion Techniques Using Bash Scripts

Shamsher khan

CEHv13 With AI (20 Modules)

Introduction to Ethical Hacking

- Overview of information security and ethical hacking concepts.
- Introduction to AI-driven ethical hacking tools.

Footprinting and Reconnaissance

- Techniques for information gathering.
- Utilization of AI for automated reconnaissance.

Scanning Networks

- Network scanning methodologies.

- AI applications in network scanning and vulnerability assessment.

Enumeration

- Extracting information from target systems.
- AI-enhanced enumeration techniques.

Vulnerability Analysis

- Identifying and analyzing vulnerabilities.
- AI-driven vulnerability assessment tools.

System Hacking

- Gaining access to systems.
- AI in password cracking and privilege escalation.

Malware Threats

- Understanding malware types and behaviors.
- AI applications in malware analysis and detection.

Sniffing

- Interception of network traffic.
- AI tools for traffic analysis and anomaly detection.

Shamsher khan

Social Engineering

- Manipulating individuals to gain unauthorized access.
- AI in detecting and preventing social engineering attacks.

Denial-of-Service (DoS) Attacks

- Techniques to disrupt services.
- AI for DoS attack detection and mitigation.

Session Hijacking

- Taking control of active sessions.
- AI methods for session monitoring and protection.

Evading IDS, Firewalls, and Honeypots

- Techniques to bypass security measures.

- AI strategies for intrusion detection and prevention.

Hacking Web Servers

- Exploiting web server vulnerabilities.
- AI tools for web server security assessment.

Hacking Web Applications

- Identifying and exploiting web application flaws.
- AI in web application security testing.

SQL Injection

- Techniques to exploit database vulnerabilities.
- AI-driven SQL injection detection and prevention.

Hacking Wireless Networks

- Exploiting wireless network weaknesses.
- AI applications in wireless network security.

Hacking Mobile Platforms

- Attacking mobile devices and applications.
- AI tools for mobile security assessment.

Shamsher khan

IoT and OT Hacking

- Securing Internet of Things and Operational Technology.
- AI in IoT and OT security.

Cloud Computing

- Understanding cloud security challenges.
- AI applications in cloud security.

Cryptography

- Principles of cryptography and encryption.
- AI in cryptographic analysis and attacks.